

我是大一，不是研一啊喂，你指望我看这么复杂的东西吗？我只能说这X mind与DeepSeek的组合技有点东西

vLLM：基于PagedAttention的高效LLM服务系统

LLM服务的内存瓶颈

- 自回归推理特性
 - 逐token生成
 - 依赖历史所有token
- KV Cache机制
 - 动态增长与收缩
 - 生命周期与长度不可预知
 - 体积巨大
 - 13B OPT模型单token约800KB
 - 2048 token序列达16GB
- 显存占用分布
 - A100 40GB上模型权重占65%
 - KV Cache占近30%
 - 占用随请求动态变化

现有系统的两大缺陷

- 内存碎片严重
 - 连续内存分配要求 — FasterTransformer、Orca等强制连续存储
 - 预分配最大长度空间
 - 导致内部碎片（实际远小于预留）
 - 导致外部碎片（多请求大小不一）
 - KV Cache有效利用率仅20.4%~38.2%
- 无法利用内存共享机会
 - 高级解码产生共享prompt的多序列
 - 并行采样
 - 束搜索
 - 独立连续分配阻断共享 — 每个序列独占KV Cache空间

吞吐量瓶颈

- LLM服务为memory-bound
 - KV Cache管理效率决定最大Batch Size
 - Batch Size直接制约吞吐量
- 算力与显存增长失衡
 - GPU算力增速快于显存容量增速
 - 内存瓶颈持续加剧

核心方法：PagedAttention

- 核心思想
 - 借鉴OS虚拟内存分页机制
 - KV Cache划分为固定大小块（Block）
 - 块物理存储无需连续
- 形式化实现
 - Kernel按块识别与获取KV
 - 不依赖连续内存布局
- 优势
 - 消除外部碎片 — 所有块大小统一
 - 减少内部碎片 — 浪费限于最后一个块
 - 支持块级共享 — 多逻辑序列可映射同一物理块

vLLM系统设计

- 整体架构
 - 集中式调度器 — 协调分布式GPU worker
 - KV Cache Manager — 分页式管理GPU物理KV Cache
 - Block Engine — 在GPU DRAM分配连续空间并划分物理块
 - Block Table
 - 维护逻辑块→物理块映射
 - 记录各逻辑块已填充token数
- 解码流程示例
 - Prompt阶段
 - 7 token prompt分配2逻辑块
 - 映射至2物理块存储KV
 - 第一次自回归解码
 - PagedAttention读取物理块
 - 新KV写入最后一块剩余额位
 - 第二次解码
 - 最后一块满，分配新逻辑块与新物理块
 - 更新Block Table
- 关键机制
 - 按需分配 — 不预分配最大长度空间
 - 块级Copy-on-Write — 共享块写入时触发复制
 - 引用计数 — 每物理块记录被引用逻辑块数

支持复杂解码场景

- 并行采样
 - 多输出共享prompt物理块
 - 生成阶段独立+Copy-on-Write
 - 内存节省
 - Alpaca: 6.1%~9.8%
 - ShareGPT: 16.2%~30.5%
- 束搜索
 - 多候选序列共享更多块
 - 共享模式动态变化
 - 内存节省
 - Alpaca: 37.6%~55.2%
 - ShareGPT: 44.3%~66.3%
 - 性能提升 — OPT-13B+Alpaca beam=6时比Orca快2.3倍
- 共享前缀
 - 多请求共享system prompt
 - 预缓存共享前缀物理块
 - 用户请求仅计算任务输入部分
 - 吞吐量提升
 - 翻译任务1-shot: 1.67倍
 - 5-shot: 3.58倍
- 混合解码方法
 - 逻辑块→物理块映射层抽象共享细节
 - 不同解码偏好请求可共批处理
 - 提升整体吞吐量

结论与贡献

- 问题识别 — 量化KV Cache内存管理对LLM服务性能的影响
- 方法创新 — 提出PagedAttention：非连续分页注意力机制
- 系统构建 — 实现vLLM：分布式LLM服务引擎，近零内存浪费
- 实验验证
 - 吞吐量提升2~4倍
 - 序列越长、模型越大、解码越复杂，增益越明显

实验评估

- 实验设置
 - 模型：OPT-13B/66B/175B、LLaMA-13B
 - 硬件：A100 40GB/80GB
 - 数据集：ShareGPT（长输入长输出）、Alpaca（短输入短输出）
 - 基线：FasterTransformer、Orca（Oracle/Pow2/Max）
- 主要结果
 - 基本采样
 - vLLM比Orca(Oracle)高1.7~2.7倍请求率
 - 比Orca(Max)高2.7~8倍
 - 比FasterTransformer高22倍
 - 并行采样与束搜索 — 序列数/beam width越大，优势越显著
 - 共享前缀 — 1-shot提升1.67倍，5-shot提升3.58倍
 - Chatbot场景 — 支持2倍于Orca的请求率
- 消融实验
 - Kernel开销
 - PagedAttention attention kernel比FasterTransformer慢20~26%
 - 端到端性能仍大幅领先
 - 块大小影响
 - 默认16：平衡GPU并行利用与碎片/共享概率
 - 过小→并行不足；过大→内部碎片↑、共享概率↓

实现与优化

- 代码规模
 - Python 8.5K行 + C++/CUDA 2K行
 - 前端：FastAPI，兼容OpenAI API
 - 支持模型：GPT、OPT、LLaMA
 - 通信：NCCL
- 关键CUDA Kernel
 - Fused reshape and block write — 融合reshape与块写入，降低kernel启动开销
 - Fusing block read and attention — 改编FasterTransformer kernel，按block table读取，支持变长
 - Fused block copy — 批量执行不连续块复制，减少小数据移动开销
- 解码算法支持
 - fork：创建新序列
 - append：追加token
 - free：删除序列
 - 可组合支持并行采样、束搜索、前缀共享

分布式执行

- Megatron-LM风格张量并行 — 每GPU worker存储部分attention head的KV Cache
- 单-KV Cache Manager — 所有worker共享统一逻辑→物理块映射
- 控制流
 - 调度器广播控制消息（token ID、block table）
 - worker执行后all-reduce同步
 - 采样token返回调度器

调度与抢占

- 调度策略
 - FCFS保证公平性
 - 抢占优先级：最早到达优先服务，最晚到达优先被抢占
- 抢占与恢复
 - All-or-Nothing原则 — 序列所有块全驱逐或全保留
 - 序列组（Sequence Group） — 同请求多序列（如beam candidates）统一调度
 - 恢复技术
 - Swapping：块交换至CPU内存，上限≤GPU KV Cache总配额
 - Recomputation：拼接已生成token+原prompt重新prefill